

THE IDEA ARCHITECT

An interruption-friendly audio companion for Claude Code

Recommendations, first steps, and a practical feature roadmap

TARGET ENVIRONMENT	CORE OUTCOME
Native macOS on a MacBook Pro M5 Max with 128 GB unified memory; Samsung Bluetooth earbuds; terminal workflow beside Claude Code.	Turn 30-90 minutes of spoken design exploration into reviewed, repo-native specifications, plans, ADRs, and actionable tasks.

RECOMMENDATION

Fork Parlor for the audio interaction layer. Keep its browser audio path, but present it as a tiny chrome-less app window. Add Opus through Bedrock only after the local loop proves comfortable for a full working session.

Executive recommendation

The shortest path to something Dee will actually use every day

Build on Parlor, but do not turn it into the entire architecture.

Parlor already contains the high-friction pieces: browser microphone capture, hands-free VAD, end-of-turn judgment, streaming TTS, barge-in, cancellation, and optional background reasoning. Fork it as the conversation runtime; keep repository workflow, model selection, and future Protocol Manager integration behind explicit adapters.

The recommended sequence

- First prove the experience: run stock Parlor on the new Mac, validate the Samsung earbud route, measure barge-in, and stay in a real conversation for 60 minutes.
- Then reduce the interface: launch a 360 x 180 px app-mode window with no tabs, address bar, or bookmark row. The terminal remains the work surface.
- Next add the senior peer: connect a Bedrock Opus provider as a background shadow architect with structured interventions and nearly verbatim speakable text.
- Only after that add repo synthesis: inspect repository conventions, draft Markdown artifacts in a staging area, and present a diff for approval before writing.
- Defer Strands and Protocol Manager integration until the product behavior is stable enough to justify durable orchestration contracts.

IMMEDIATE

Make it feel natural

- Fork + baseline
- Tiny app window
- 60-minute session

NEAR TERM

Make it useful

- Opus shadow
- Repo context
- Reviewed artifacts

NICE TO HAVE

Make it powerful

- Protocol Manager
- Native shell
- Team workflows

The central product test

Can it stay out of your way for an hour?

Latency demos are not enough. The prototype succeeds when you can brainstorm while coding, interrupt naturally, park a thread, recover it later, and finish with documentation you trust more than a raw transcript.

Target experience and boundaries

What this product is - and what it is not

The Idea Architect is a local-first conversational design companion launched from a repository. It is not a replacement for Claude Code, a general voice assistant, or a meeting recorder. Its job is to make spoken enterprise engineering design productive and durable.

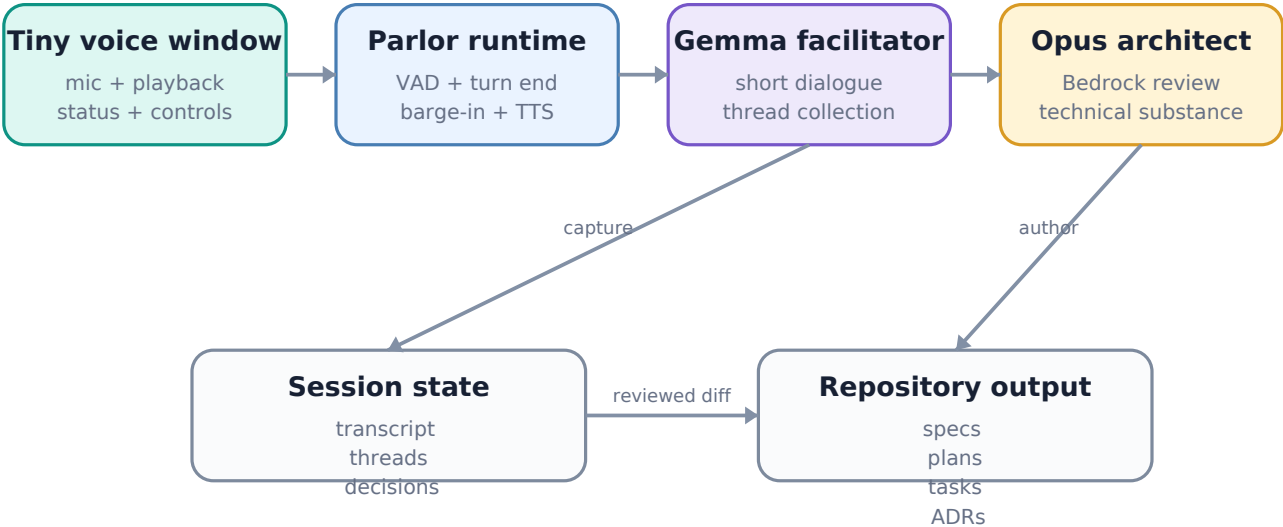
Owns	Does not own
Continuous listening, turn detection, interruption, and spoken pacing	Autonomous code implementation
Thread capture, questions, challenges, and background consultation	Unreviewed repository writes
Session summaries and proposed repo-native artifacts	Production effects or deployment authority
A lightweight audio/status window plus terminal control	A large browser workspace

Model roles

Component	Responsibility	Guardrail
Gemma 4	Fast facilitator: listen, acknowledge, ask safe clarifiers, track conversational motion.	Must not independently author complex architectural conclusions.
Opus on Bedrock	Shadow architect: technical interpretation, contradiction detection, alternatives, deep dives, and artifact substance.	Returns structured interventions with a speakable field and supporting detail.
Claude Code	Reads the approved artifacts and performs implementation and validation in the normal workflow.	Starts from explicit tasks and acceptance criteria; no transcript reconstruction.
Protocol Manager (later)	Durable evidence, evaluation, controlled workflow, and external-effect boundaries.	Not required to prove the audio product.

Recommended architecture

Parlor remains replaceable; the session and repository contracts remain yours



Key interfaces

Boundary	Minimum contract
Audio runtime	start, pause, resume, stop, select devices, audio/transcript events, interruption event
Conversation model	recent context + session state -> streamed speech text + candidate actions
Architect provider	thread snapshot + repo context -> structured intervention or synthesis
Session store	append-only transcript/events plus mutable thread summary and confirmed decisions
Repo adapter	discover conventions, propose files, build patch, validate scope, apply approved selections

Why not Strands first?

Strands can become a useful provider later, especially for standardized events, Bedrock agents, hooks, and AgentCore deployment. But Parlor already has the exact experiential loop under test. Introducing a second orchestration framework before validating the audio experience adds adapters and failure modes without improving the first user outcome.

Contain the fork

Keep upstream Parlor code recognizable. Add new behavior in provider, session, repository, and UI modules. Avoid rewriting the working audio path until measurements or reliability tests prove it necessary.

First steps: prove the interaction

Phase 0 and Phase 1 - roughly the first 3 to 5 focused development days

Phase 0 - baseline

Step	Action	Acceptance criterion
0.1	Fork Parlor and pin the upstream commit, Python environment, llama.cpp build, and model assets.	A clean setup script brings up the stock app on the M5 Max from a new checkout.
0.2	Run the included latency and turn-detection benchmarks before modifying code.	Baseline results are committed under a clearly dated benchmark folder.
0.3	Validate audio routing: Samsung mic + Samsung output; Mac mic + Samsung output; Mac mic + Mac speakers.	Each route completes 20 turns; device failure is logged and recoverable.
0.4	Complete a real 60-minute brainstorming session using stock behavior.	No crash; transcript preserved; interruption failures and fatigue points recorded.

Phase 1 - tiny focused surface

Step	Action	Acceptance criterion
1.1	Create a compact app-mode route, approximately 360 x 180 px.	No browser chrome; shows listening, thinking, speaking, paused, timer, and latest transcript line.
1.2	Add terminal launcher from the active repo.	One command starts backend and window, records repo root, and shuts both down cleanly.
1.3	Add explicit mute, pause, resume, and end-session controls plus keyboard shortcuts.	Controls work during listening and playback; end-session flushes state before exit.
1.4	Persist raw transcript, model outputs, timing, and interruption events locally.	An interrupted answer remains distinguishable from a completed answer.

Immediate roadmap

Make it natural before making it clever

Capability	Implementation guidance	Done when
Reliable barge-in	Keep microphone active during TTS; abort server generation and queued audio; tag output as interrupted.	Speech stops perceptibly fast and false barge-ins are rare across 50 mixed turns.
Turn completion	Keep Silero VAD plus smart-turn-v3; tune silence thresholds from measured sessions, not preference guesses.	Mid-thought pauses are held; completed thoughts do not wait awkwardly.
Bluetooth routing	Prefer Samsung duplex. Observe disconnects and unusable streams; reopen audio and fall back to Mac mic only on failure.	A fallback event does not lose more than the current utterance.
Long-session state	Append events continuously; roll active context into recent turns + session summary + open/parked thread index.	A 90-minute session does not require replaying the entire transcript to the model.
Conversation posture	Use the interview script as a prompt policy, not a rigid questionnaire. Allow explore, challenge, summarize, and just-listen modes.	The facilitator does not force every session through every scripted question.
Session recovery	Write state atomically and reopen incomplete sessions.	A process restart recovers transcript, current thread, and parked threads.

Immediate release gate

Do not integrate Opus until this version is comfortable for one full hour on three separate days. Audio defects become harder to isolate once background reasoning, repository inspection, and artifact generation are also active.

Near-term roadmap

Make the conversation technically useful and repo-native

Capability	Recommended shape	Primary safeguard
Bedrock Opus shadow architect	Send transcript deltas plus thread summaries asynchronously. Support background, next-boundary, and interrupt urgency.	Gemma speaks important findings nearly verbatim from an Opus-authored speakable field.
Structured interventions	Use typed question, challenge, correction, finding, deep-dive offer, and synthesis messages.	Every intervention carries affected thread IDs and provenance.
Repository discovery	Read AGENTS.md, CLAUDE.md, README, docs/specs/plans/tasks/ADR conventions, branch, and dirty state.	Read-only by default; no broad scan of ignored, secret, or generated paths.
Session synthesis	Maintain problem, outcome, constraints, decisions, rejected alternatives, open questions, risks, tasks, and acceptance criteria.	Confirmed versus inferred items are visibly distinct.
Artifact drafting	Let Opus author Markdown into a staging directory matching the repository's conventions.	Every proposed file has a source thread and purpose.
Diff review and apply	Show a terminal diff; allow select, edit, reject, or apply. Preserve unrelated dirty worktree changes.	No file write occurs without explicit approval of the proposed patch.
Claude Code handoff	Produce a concise starting prompt referencing approved artifact paths and first task.	Claude Code can begin implementation without access to the full audio transcript.
Observability	Emit timings for VAD, turn-end, first model token, first audio, architect latency, interruptions, and session recovery.	A session report makes experience regressions diagnosable.

Nice-to-have roadmap

Add power only where sustained use proves the need

Feature	Value	Why it waits
Protocol Manager integration	Promote confirmed constraints, decisions, and evidence into existing typed protocol flows and NodeEvidence.	Promotion rules and citations deserve separate design after the interview loop is stable.
Strands provider	Standardize Bedrock agents, event hooks, and future AgentCore deployment behind an orchestration adapter.	The current bidirectional APIs are not needed to validate Parlor's local audio experience.
Native Swift or Tauri shell	True menu-bar presence, global shortcuts, better device monitoring, and a polished floating voice puck.	Chrome app mode is much cheaper and sufficient for behavioral validation.
Screen and selection context	Attach the current terminal output, selected code, diagram, or diff to an active thread.	Privacy, scope, and token-budget rules must be explicit first.
Voice commands for thread operations	Open, name, park, resume, merge, and summarize threads hands-free.	Thread semantics should emerge from actual sessions before commands are frozen.
Multi-model review panel	Run architect, critic, security, and domain-specialist providers in parallel for high-stakes designs.	Cost and synthesis complexity are unjustified for the MVP.
Repo workflow plugins	Adapters for GitHub issues, Linear, Jira, or organization-specific task schemas.	Start with portable Markdown and learn the common denominator.
Team session mode	Multiple speakers, attribution, shared review, and decision confirmation.	The first product is a personal companion; multi-user audio changes the problem substantially.
Semantic session retrieval	Find prior decisions and similar threads across repositories and sessions.	Requires strong privacy, identity, retention, and supersession semantics.

Deliberate non-goals for the first release

- Autonomous implementation, deployment, or external effects.
- Perfect full-duplex speech-to-speech behavior; good cascade behavior is sufficient.
- A universal documentation schema imposed on every repository.
- A cloud service or team product before the local personal workflow proves durable.

Validation plan and product metrics

Measure conversational usefulness, not just token speed

Dimension	Metric	Initial target
Responsiveness	End of utterance to first audible response	Median under 1.5 s for lightweight Gemma turns on target hardware
Barge-in	Speech onset to playback stop	P95 under 300 ms
Turn quality	False end / missed end events	Below 5% across a labeled 100-turn set
Stability	Unrecovered crashes or audio deadlocks	Zero in a 90-minute session
Continuity	Correct recovery of named or parked threads	At least 9 of 10 scripted retrieval checks
Architect value	Interventions rated useful, neutral, or disruptive	Useful >60%; disruptive <10% after tuning
Artifact fidelity	Confirmed decisions correctly represented	100% after review; no unmarked invention
Workflow leverage	Time from session end to approved repo artifacts	Under 10 minutes for a typical design session

Three release gates

Gate	Evidence required
G1 - Natural	Three separate 60-minute local sessions; acceptable interruption, turn-taking, routing, and fatigue.
G2 - Insightful	Opus interventions improve design discussion without hijacking its flow; structured provenance remains intact.
G3 - Durable	Approved artifacts match repository conventions and can drive Claude Code implementation without transcript access.

The kill criterion

If the compact Parlor fork cannot remain stable and pleasant for an hour after targeted tuning, stop layering features. Re-evaluate the audio runtime or replace only that adapter; do not bury the defect under more orchestration.

Suggested first two weeks

A focused sequence for a credible personal alpha

Window	Focus	Concrete output
Days 1-2	Baseline and hardware	Pinned fork; reproducible setup; benchmark results; device matrix; 60-minute session notes
Days 3-4	Tiny interface and launcher	App-mode voice window; terminal command; clean startup/shutdown; status and controls
Day 5	Persistence and recovery	Session event log; transcript; interrupted-output marker; crash recovery test
Days 6-7	Interview behavior	Technical-peer prompt; explore/challenge/summarize/listen postures; thread capture draft
Days 8-9	Opus provider spike	Bedrock ConverseStream adapter; structured intervention schema; one background and one next-boundary flow
Day 10	Personal alpha review	End-to-end 60-minute design session; metric report; go/no-go decision for repo synthesis

Recommended repository shape

Area	Purpose
src/audio	Parlor-compatible audio and browser transport adapters
src/conversation	Facilitator, turn events, thread state, and session lifecycle
src/architect	Bedrock Opus provider and structured intervention contracts
src/repository	Context discovery, convention detection, staging, diff, and apply
web/compact	Tiny app-mode status and control surface
benchmarks	Upstream baseline plus project-specific long-session and barge-in tests
docs/decisions	Project ADRs, including why Parlor, why browser audio, and why deferred Strands

First implementation decision

Start with Parlor's current E4B default for conversational quality, but benchmark E2B and 12B on the M5 Max. Choose from measured response rhythm and transcript quality; the machine has enough memory that model size is not the limiting question.

References and current-source notes

Verified 9 August 2026

[1] Parlor repository and README. Current architecture, requirements, benchmark commands, configuration, background reasoner, barge-in, VAD, smart-turn-v3, browser transport, Gemma 4, and Kokoro details. <https://github.com/fikrikarim/parlor>

[2] Google AI for Developers, Gemma audio understanding. Current audio support, model IDs, and audio limits. <https://ai.google.dev/gemma/docs/capabilities/audio>

[3] Amazon Bedrock, Converse API. Provider-neutral conversational inference and streaming operations. <https://docs.aws.amazon.com/bedrock/latest/userguide/conversation-inference.html>

[4] Amazon Bedrock, ConverseStream with Anthropic Claude. Streaming response examples for Python and other SDKs. [AWS ConverseStream Claude example](#)

[5] Claude Code documentation, project memory and CLAUDE.md behavior. <https://docs.anthropic.com/en/docs/claude-code/memory>

[6] Claude Code hooks reference. Lifecycle hooks can later connect deterministic repo or session actions. <https://docs.anthropic.com/en/docs/claude-code/hooks>

[7] Claude Code overview. Terminal workflow, repository access, editing, and tool execution. <https://docs.anthropic.com/en/docs/claude-code/overview>

Assumptions used

- The target is a personal, local-first tool for Dee's normal terminal-based Claude Code workflow.
- The first hardware target is a MacBook Pro M5 Max with 128 GB unified memory and Samsung Bluetooth earbuds.
- Opus is accessed through Amazon Bedrock rather than Anthropic's native voice product.
- Repository writes remain reviewed and explicit; implementation stays in Claude Code.
- Protocol Manager and its NodeEvidence model are available later but are intentionally outside the first audio milestone.

Bottom line

Parlor appears to be the right starting point because it has already solved most of the interaction loop you were searching for. The strongest roadmap is disciplined: first natural, then insightful, then durable, then powerful.